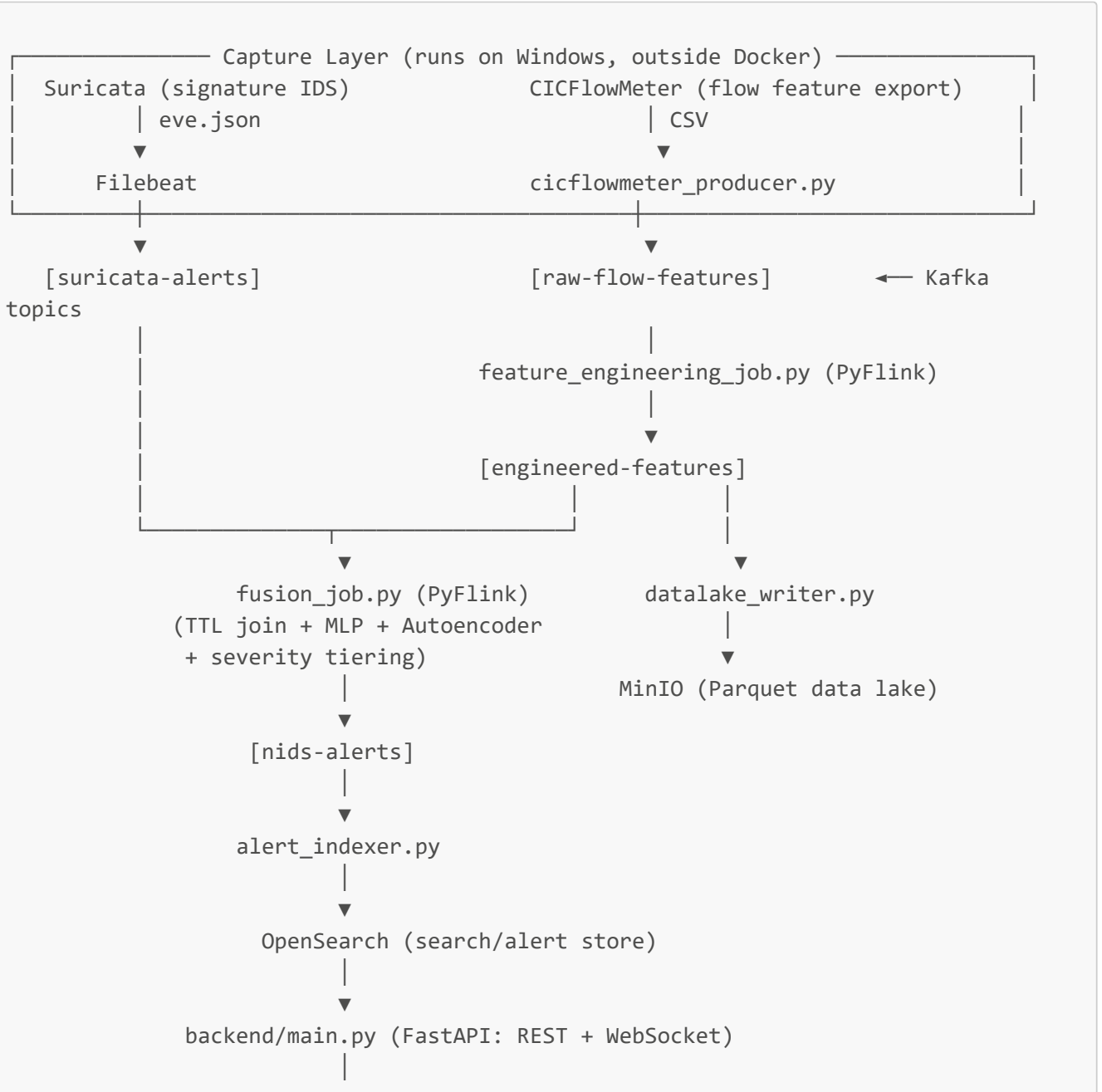
# Hybrid NIDS — Kafka + Flink + Data Lake + Search/Alerting Backend

A real-time Network Intrusion Detection System that fuses **signature-based detection** (Suricata) with **machine-learning anomaly detection** (an MLP classifier + an Autoencoder) inside an **Apache Flink** streaming pipeline, then archives and serves the results through a **MinIO/Parquet data lake**, an **OpenSearch** search/alerting store, and a **FastAPI** backend.

This README documents the system as a whole: what each piece does, the full folder structure, how to set up the Windows-side tools that live outside Docker, and how to start/stop everything with one command.

---

## 1. Architecture



```
                              ▼
                Frontend / SOC dashboard / curl / Swagger UI
```

**Why two storage backends?** They serve different jobs. MinIO/Parquet is cheap, durable, columnar storage for *every* engineered feature vector — built for offline analysis, retraining, and auditing, not for fast lookups. OpenSearch holds only the *final fused alerts* — built for instant search, filtering, and aggregation ("show me CRITICAL alerts from this source IP in the last hour"). Kafka decouples every stage from every other stage: if one consumer is slow or restarting, nothing else in the pipeline blocks.

---

## 2. Full Folder Structure

```
NIDS_with_kafka_and_flink/
├── docker-compose.yml            Orchestrates all 11 containers (below)
├── Dockerfile.flink               Flink 1.18 image: PyFlink + TensorFlow +
scikit-learn + Kafka connector jar
├── Dockerfile.backend             Shared image for backend / datalake-writer /
alert-indexer (FastAPI, boto3, opensearch-py, pyarrow)
├── requirements.txt               Deps for the Flink image + host-side
producer/notebook cell
├── requirements-backend.txt       Deps for backend + the two storage services
├── start-pipeline.ps1             ⭐ Starts EVERYTHING in the correct order
(see §6)
├── stop-pipeline.ps1              ⭐ Stops EVERYTHING cleanly (see §6)
├── .gitignore                     Excludes venv/, __pycache__/, build output,
local logs
├── README.md                      This file
│
├── flink_jobs/                    The two PyFlink streaming jobs
│   ├── common.py                    Shared Kafka topic names + flow_key() join-
key format
│   ├── feature_engineering_job.py   Job 1: cleans/orders/imputes flow features
using training-time medians
│   └── fusion_job.py                Job 2: keyed TTL join of alerts+features,
MLP+Autoencoder scoring, severity tiering
│
├── services/                      Storage-layer Python microservices (Layer 5)
│   ├── storage_common.py            Shared config: Kafka/MinIO/OpenSearch
settings, all env-var overridable
│   ├── datalake_writer.py           Kafka (engineered-features) → batches →
Parquet → MinIO
│   └── alert_indexer.py             Kafka (nids-alerts) → bulk-indexed into
OpenSearch (daily-rolling index)
│
├── backend/                       Query/serving API (Layer 5)
│   └── main.py                      FastAPI: GET /alerts, /alerts/{id},
/stats/severity, /datalake/objects, WS /ws/alerts
│
├── producers/                     Bridges external tools into Kafka
│   └── cicflowmeter_producer.py     Tails CICFlowMeter's live CSV, publishes
each flow row to raw-flow-features
```

```
│
├── filebeat/                        Config for shipping Suricata alerts into Kafka
│   └── filebeat.yml                   Tails eve.json, filters to event_type=alert,
ships to suricata-alerts
│
├── scripts/
│   ├── create_topics.sh               Creates the 4 Kafka topics (Linux/Git Bash -
see §6 for the PowerShell equivalent)
│   └── save_training_metadata.py      Notebook cell: generates
feature_columns.json / feature_medians.json
│
└── models/                           Trained model artifacts (populate this
yourself - see §5)
    ├── neural_network_ids.h5, scaler.pkl, label_encoder.pkl        (from
nids_mlp_pipeline.ipynb)
    ├── feature_columns.json, feature_medians.json                  (from
save_training_metadata.py)
    └── nids_autoencoder.h5, ae_scaler.pkl, ae_threshold.pkl         (from
nids_autoencoder_train.ipynb)
```

**Not in the repo, created locally per machine (see §3–§4):**

```
C:\suricata-logs\                 Suricata's eve.json + stats.log (writable path,
not under Program Files)
C:\filebeat-data\                 Filebeat's registry/state (writable path, avoids
Program Files permission errors)
C:\CICFlowMeter-app\...\data\daily\   CICFlowMeter's live CSV output (created
automatically when it starts capturing)
venv\                             Python virtual environment (never share/commit
this - rebuild per machine)
```

### What runs where

| **Runs in Docker (`docker-compose.yml`)** | **Runs directly on Windows** |
|---|---|
| Kafka, Kafka UI, Flink JobManager/TaskManager | Suricata |
| MinIO, MinIO bucket-init | Filebeat |
| OpenSearch, OpenSearch Dashboards | CICFlowMeter |
| `backend`, `datalake-writer`, `alert-indexer` | `cicflowmeter_producer.py` (venv) |

Docker Desktop only ever sees Kafka traffic — the network-facing tools (Suricata's packet capture, Npcap, CICFlowMeter's capture) need real interface access, which is why they run natively on Windows instead of inside a container.

---

## 3. One-Time Windows Setup (outside Docker)

These are the tools Docker Compose can't manage for you. Each one had a specific gotcha discovered while building this - noted inline so you don't have to rediscover them.

### 3.1 Suricata

1. Install from https://suricata.io/download/ — during setup, install **Npcap** with "WinPcap API-compatible Mode" ticked.
2. Create a writable log directory (Suricata can't write to its own `Program Files` install folder without elevation):

   ```
   mkdir C:\suricata-logs
   ```

   Edit `suricata.yaml` → set `default-log-dir: C:\suricata-logs\`.
3. Point `rule-files:` in `suricata.yaml` at rules that actually exist — the default list references Emerging Threats rule files that aren't bundled with the installer. Either download real ET rules, or create a `local.rules` file with your own test rules under `C:\Program Files\Suricata\rules\`.
4. **Run with your adapter's IP address, not its friendly name.** Running `-i Wi-Fi` (or any interface *name*) crashes this Suricata Windows build with an access violation (`0xC0000005`) — a known issue. Use the IP instead:

   ```
   Get-NetIPAddress -InterfaceAlias "Wi-Fi" -AddressFamily IPv4   # find your IP
   cd "C:\Program Files\Suricata"
   .\suricata.exe -i <your-ip-address> -c suricata.yaml -vvv
   ```

   (Admin PowerShell required.)

### 3.2 Filebeat

1. Download from https://www.elastic.co/downloads/beats/filebeat.
2. This project's `filebeat/filebeat.yml` already uses the current `filestream` input (the older `log` input type was removed in Filebeat 9.x) and points `output.kafka.hosts` at `localhost:29092` (Kafka's host-reachable listener, since Filebeat runs outside Docker).
3. Create a writable data directory (writing to `Program Files\...\filebeat\data` fails with "Access is denied" unless run elevated):

   ```
   mkdir C:\filebeat-data
   ```

4. Run **elevated** (Admin PowerShell), pointing at both the writable data path and this repo's config:

```
cd "C:\Program Files\Elastic\Beats\9.4.4\filebeat"
.\filebeat.exe -e -c
"D:\path\to\NIDS_with_kafka_and_flink\filebeat\filebeat.yml" --path.data
"C:\filebeat-data"
```

### 3.3 CICFlowMeter

CICFlowMeter has no Windows binary release — it's built from source.

1. Install **JDK 8** specifically (separate from whatever newer JDK you may already have — the build tooling requires it) and **Maven**.
2. Clone: `git clone https://github.com/ahlashkari/CICFlowMeter.git`
3. Install the bundled `jnetpcap` native library into your local Maven repo (exact version/path may differ — check `jnetpcap\win\`):

   ```
   mvn install:install-file -Dfile=jnetpcap\win\jnetpcap-1.4.r1425\jnetpcap.jar
   -DgroupId=org.jnetpcap -DartifactId=jnetpcap -Dversion=1.4.1 -Dpackaging=jar
   ```

   If this fails with a corrupted-path error, put the command in a `.ps1` script file instead of typing/pasting it directly — PowerShell can mangle dots in inline arguments.
4. Build: `.\gradlew.bat distZip` (several minutes the first time).
5. Extract the resulting zip from `build\distributions\`. Its `data\daily` subfolder (under wherever you extracted it, alongside `CICFlowMeter.bat`) is where live flow CSVs land — that's the `--watch-dir` the producer needs in §6.
6. Run **elevated**: `.\CICFlowMeter.bat`, pick your interface from the list, click **Start**.

### 3.4 Python environment

```
cd "D:\path\to\NIDS_with_kafka_and_flink"
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If `Activate.ps1` is blocked: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`.

---

## 4. One-Time: Populate `models/`

The Flink jobs need 8 files that aren't produced by default — they can't safely guess column order or impute a missing streaming value without them.

1. Open `nids_mlp_pipeline.ipynb`. Paste `scripts/save_training_metadata.py` as a new cell right after **Step 4** (train/test split), before **Step 5** (scaling). Run it → produces `feature_columns.json` and `feature_medians.json`.

2. Run Step 10 of that notebook → produces `neural_network_ids.h5`, `scaler.pkl`, `label_encoder.pkl`.
3. Run Step 8 of `nids_autoencoder_train.ipynb` → produces `nids_autoencoder.h5`, `ae_scaler.pkl`, `ae_threshold.pkl`.
4. Copy all 8 into this project's `models/` folder.

---

## 5. Docker Compose Services (11 total)

| Service | Image / build | Purpose | Host port(s) |
|---|---|---|---|
| `kafka` | apache/kafka:3.7.0 | Message broker (KRaft mode, no Zookeeper) | 9092, 29092 |
| `kafka-ui` | provectuslabs/kafka-ui | Browse topics/messages | 8080 |
| `jobmanager` / `taskmanager` | `Dockerfile.flink` | Flink cluster running the 2 streaming jobs | 8081 |
| `minio` | minio/minio | S3-compatible object storage (data lake) | **9010**, **9011** (console) |
| `minio-init` | minio/mc | One-shot: creates the `nids-datalake` bucket, then exits | — |
| `opensearch` | opensearchproject/opensearch:2.15.0 | Alert search/log store | 9200 |
| `opensearch-dashboards` | opensearchproject/opensearch-dashboards | Visual query/dashboard UI | 5601 |
| `datalake-writer` | `Dockerfile.backend` | Kafka → Parquet → MinIO | — |
| `alert-indexer` | `Dockerfile.backend` | Kafka → OpenSearch | — |
| `backend` | `Dockerfile.backend` | FastAPI REST + WebSocket API | 8000 |

> **Note on MinIO's ports:** mapped to **9010/9011** on the host, not the conventional 9000/9001 — on Windows, port 9000 frequently falls inside a Hyper-V/WSL reserved port range and fails to bind (`bind: An attempt was made to access a socket in a way forbidden by its access permissions`). This only affects the *host-exposed* port; every container-to-container connection still uses `minio:9000` internally, so nothing else needed to change.

> **OpenSearch prerequisites baked into the compose file:** `ulimits` (`memlock`/`nofile`) and `OPENSEARCH_INITIAL_ADMIN_PASSWORD` are both required for the container to start at all — even with `plugins.security.disabled: true`, OpenSearch 2.12+'s bootstrap script refuses to start without a password env var present, and its bootstrap checks fail without the ulimits. Both are set to dev-only values already.

---

# 6. Running the System

## 6.1 The easy way — `start-pipeline.ps1` / `stop-pipeline.ps1`

Edit the path variables at the top of `start-pipeline.ps1` once (project directory, Suricata's install dir + your IP, Filebeat's install dir, CICFlowMeter's extracted dir) to match your machine, then:

```
cd "D:\path\to\NIDS_with_kafka_and_flink"
powershell -ExecutionPolicy Bypass -File ".\start-pipeline.ps1"
```

**What it does, in order:**

1. `docker compose up -d --build` — all 11 containers (rebuilds images if source changed)
2. Creates the 4 Kafka topics (idempotent — safe to re-run)
3. Fixes Flink checkpoint directory permissions inside the containers
4. Launches Suricata in an **elevated** window (UAC prompt — click Yes)
5. Launches Filebeat in an **elevated** window (UAC prompt — click Yes)
6. Launches CICFlowMeter's GUI in an **elevated** window (UAC prompt — click Yes; **you still click Start manually** in the GUI, that part can't be automated)
7. Activates the venv and launches the flow producer
8. Submits both Flink jobs in **detached** mode (`-d` — without it, the CLI blocks forever on a streaming job and the second job would never get submitted)
9. Prints a process-check summary and every dashboard URL

To stop everything:

```
powershell -ExecutionPolicy Bypass -File ".\stop-pipeline.ps1"
```

This cancels the Flink jobs, prompts you to Ctrl+C the Suricata/Filebeat/ CICFlowMeter/producer windows (they can't be closed remotely), then runs `docker compose down` — **all data survives** (Kafka topics, Flink checkpoints, MinIO Parquet files, OpenSearch indices all live in Docker volumes). Use `docker compose down -v` instead if you want a full wipe.

## 6.2 The manual way (what the script automates)

```
docker compose up -d --build

docker exec nids-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --if-not-exists --topic suricata-alerts --partitions 3 --
replication-factor 1
docker exec nids-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --if-not-exists --topic raw-flow-features --partitions 3 -
-replication-factor 1
docker exec nids-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --if-not-exists --topic engineered-features --partitions 3
--replication-factor 1
```

```
docker exec nids-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --if-not-exists --topic nids-alerts --partitions 3 --
replication-factor 1

docker compose exec -u root jobmanager chmod -R 777 /opt/flink/checkpoints
docker compose exec -u root taskmanager chmod -R 777 /opt/flink/checkpoints

# In separate elevated windows: Suricata, Filebeat, CICFlowMeter (§3)

.\venv\Scripts\Activate.ps1
python producers/cicflowmeter_producer.py --watch-dir "<CICFlowMeter's data\daily
path>" --bootstrap-servers localhost:29092

docker compose exec jobmanager flink run -d -pyFiles
/opt/flink/usrlib/flink_jobs/common.py -py
/opt/flink/usrlib/flink_jobs/feature_engineering_job.py
docker compose exec jobmanager flink run -d -pyFiles
/opt/flink/usrlib/flink_jobs/common.py -py
/opt/flink/usrlib/flink_jobs/fusion_job.py
```

### 6.3 Dashboards

| What | URL |
|---|---|
| Kafka UI | http://localhost:8080 |
| Flink dashboard | http://localhost:8081 |
| MinIO console | http://localhost:9011 (`minioadmin` / `minioadmin`) |
| OpenSearch | http://localhost:9200 |
| OpenSearch Dashboards | http://localhost:5601 |
| Backend API (Swagger UI) | http://localhost:8000/docs |

---

## 7. Using the Backend API

```
# PowerShell's `curl` is aliased to Invoke-WebRequest and will nag about
# script execution - use Invoke-RestMethod instead for clean JSON:
Invoke-RestMethod "http://localhost:8000/alerts?since_minutes=60&limit=20"
Invoke-RestMethod "http://localhost:8000/alerts?
severity_tier=HIGH&since_minutes=1440"
Invoke-RestMethod "http://localhost:8000/alerts/<doc_id>"
Invoke-RestMethod "http://localhost:8000/stats/severity?since_minutes=1440"
Invoke-RestMethod "http://localhost:8000/datalake/objects?date=2026-08-07"
```

Live feed from a browser console:

```
const ws = new WebSocket("ws://localhost:8000/ws/alerts");
ws.onmessage = (e) => console.log(JSON.parse(e.data));
```

## 8. Querying the Data Lake Directly (DuckDB, outside the API)

```
INSTALL httpfs; LOAD httpfs;
SET s3_endpoint='localhost:9010';
SET s3_access_key_id='minioadmin';
SET s3_secret_access_key='minioadmin';
SET s3_url_style='path';
SET s3_use_ssl=false;

SELECT * FROM read_parquet('s3://nids-datalake/engineered-features/dt=2026-08-07/**/*.parquet')
LIMIT 20;
```

---

## 9. Tuning Knobs

- `FUSION_TTL_SECONDS` in `fusion_job.py` — how long an alert or feature record waits in Flink keyed state for a match before eviction.
- `determine_severity()` in `fusion_job.py` — CRITICAL/HIGH/MEDIUM/LOW thresholds; a starting point, tune to your risk model.
- `DATALAKE_BATCH_MAX_ROWS`/`_SECONDS` (env vars on `datalake-writer`) — Parquet file size vs. write latency trade-off.
- `ALERT_INDEXER_BATCH_MAX_DOCS`/`_SECONDS` (env vars on `alert-indexer`) — same trade-off for OpenSearch bulk-indexing.
- `parallelism.default` / `taskmanager.numberOfTaskSlots` — scale with `docker compose up --scale taskmanager=2 -d` if either Flink job shows backpressure.
- `plugins.security.disabled` / **MinIO's hardcoded credentials** — both dev-only. Re-enable OpenSearch auth/TLS and rotate MinIO credentials before this touches anything beyond your local machine.

---

## 10. Troubleshooting Reference

Problems actually hit while building this, in case they recur:

| Symptom | Cause | Fix |
|---|---|---|
| Suricata exits with code `-1073741819` | Running with interface *name* (`-i Wi-Fi`) | Use the interface's IP address instead |
| Filebeat: "Access is denied" on its data path | Writing to `Program Files` without elevation | Run elevated + `--path.data "C:\filebeat-data"` |

| Symptom | Cause | Fix |
|---|---|---|
| Filebeat: "Log input is deprecated" | Config used the old `log` input | `filebeat.yml` here already uses `filestream` |
| `flink run` blocks forever, 2nd job never submits | Streaming jobs run attached by default | Add `-d` (detached) |
| `ModuleNotFoundError: No module named 'flink_jobs'` | Import path issue in the job scripts | Ship `common.py` explicitly via `-pyFiles` |
| `Failed to create directory for shared state` (Flink) | Checkpoint dir owned by root inside container | `docker compose exec -u root <svc> chmod -R 777 /opt/flink/checkpoints` |
| OpenSearch container exits immediately | Missing `OPENSEARCH_INITIAL_ADMIN_PASSWORD` (2.12+) or missing `ulimits` | Both already set in `docker-compose.yml` |
| `docker compose up` fails: "ports are not available" on 9000 | Windows Hyper-V/WSL reserved port range | MinIO already remapped to host port 9010/9011 |
| Every alert fails to index: `mapper_parsing_exception` on `@timestamp` | CICFlowMeter's raw timestamp format (`DD/MM/YYYY hh:mm:ss AM/PM`) isn't ISO-8601 | `alert_indexer.py`'s `normalize_timestamp()` already handles this |
| `mvn install:install-file` fails with a mangled path | Copy-pasted command corrupted by shell quoting | Put the command in a `.ps1` script file and run that instead |